

BT-GS Solver

A Quick Guide

Morteza Maleknia ^{*}
Majid Soleimani-Damaneh [†]

September 6, 2026

Abstract

This guide provides a concise introduction to the **Bundle-Trust-Gradient-Sampling (BT-GS)** solver, a robust descent method for minimizing locally Lipschitz functions over $\mathbb{R}^n$. It describes the class of problems addressed by the solver, installation requirements, basic usage, input and output formats, available algorithmic parameters, and representative examples. The solver has been implemented in MATLAB and is based on the results presented in the following research paper:

Morteza Maleknia and Majid Soleimani-damaneh, A bundle-trust method via gradient sampling technique for nonsmooth optimization using exact and inexact gradients, IMA Journal of Numerical Analysis, 2025, draf087. <https://doi.org/10.1093/imanum/draf087>

Contents

1	Introduction	2
2	Requirements	2
3	Installation and a Quick Test	3
4	Inputs of the BT-GS	3
5	Output of the BT-GS	4
6	Example 1	4
7	Options and Stopping Criteria	5
8	Example 2	7
9	Smooth Minimization Problems	8
10	Troubleshooting	8
11	Citation	8
12	Contact	8

^{*}Department of Mathematical Sciences, Isfahan University of Technology, Isfahan 84156-83111, Iran

[†]School of Mathematics, Statistics and Computer Science, College of Science, University of Tehran, Tehran, Iran

1 Introduction

The **BT-GS** is a robust **first-order** solver designed for solving optimization problems of the form

$$\min_{x \in \mathbb{R}^n} f(x), \quad (1)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a locally Lipschitz function. The solver is particularly intended for problems where the objective function is **nonsmooth** and possibly **nonconvex**. It is also recommended for solving **ill-conditioned smooth optimization problems**. The solver has been developed in MATLAB based on the theoretical and algorithmic framework presented in the following research article:

Morteza Maleknia and Majid Soleimani-damaneh, "A bundle-trust method via gradient sampling technique for nonsmooth optimization using exact and inexact gradients," IMA Journal of Numerical Analysis, 2025, draf087. <https://doi.org/10.1093/imanum/draf087>

The main features of the **BT-GS** solver are summarized as follows:

- It is a descent method.
- It operates using function and subgradient evaluations.
- It supports three main types, depending on the structure of the objective function:
 - Type 0:** For **nonconvex** objective functions, or when **the user has no prior knowledge** of the structure of the objective function f .
 - Type 1:** For **convex** objective functions.
 - Type 2:** For **convex piecewise linear** objective functions.
- It supports three modes for supplying subgradients:
 - Mode 0:** The subgradients are automatically approximated from the f -values using a forward-difference formula with a constant step size. Thus, **the user only needs to supply the function values**.
 - Mode 1:** The subgradients are automatically approximated from the f -values using a forward-difference formula with a decaying step-size rule. Thus, **the user only needs to supply the function values**.
 - Mode 2:** The user directly supplies valid subgradients of the objective function. Thus, **the user provides both function values and subgradients**. This is the *default mode* of the **BT-GS** and is generally the *most robust* and *fastest* mode.

Notice: Under Modes 1 and 2, the convergence of the **BT-GS** to a Clarke stationary point is theoretically supported.

2 Requirements

The current implementation requires:

- Windows, Linux, or macOS as the operating system.

- MATLAB software, preferably R2021 or later, with the **Optimization Toolbox**, which provides access to the built-in `quadprog` solver.
- Sufficient memory to store the problem data and intermediate quantities.

3 Installation and a Quick Test

1. Download the BT-GS.zip from one of the following links:

Link 1. <https://iutbox.iut.ac.ir/index.php/s/3Ea6BEGLYS3a6Sb>

Link 2. <https://people.iut.ac.ir/en/maleknia/content/1706421>¹

2. Unzip the file and copy the BT-GS folder in an arbitrary directory.
3. Open the BT-GS folder.
4. Open **Test.m** in MATLAB and run it.
5. If the run is successful, a table of results should appear in the **Command Window**. This verifies that the code is consistent with your MATLAB software. Check also the output structure from the **Workspace**.

Notice: If you have any difficulty regarding this quick test, including consistency of the code, unexpected errors, or anything else, please do not hesitate to contact us (see Section 12).

4 Inputs of the BT-GS

The **BT-GS** solver can be called using

```
output = BTGS(@fun, x0, Type, options);
```

As shown above, the solver has four inputs:

1. **fun** is a MATLAB function that, for a given $\mathbf{x} \in \mathbb{R}^n$, returns the function value $f(\mathbf{x}) \in \mathbb{R}$. Depending on the selected mode for supplying subgradients, it may also return an arbitrary subgradient $\mathbf{g} \in \partial f(\mathbf{x}) \subset \mathbb{R}^n$. This is a *compulsory* input.
2. $\mathbf{x}_0 \in \mathbb{R}^n$ is the starting point of the **BT-GS** solver. The solver uses the size of this input to determine the dimension of the problem. It can be either a row or a column vector. This is a *compulsory* input.
3. **Type** is an integer input parameter that must be chosen from the set $\{0, 1, 2\}$, indicating the structure of the objective function. This is a *compulsory* input and is set as follows:
 - Set **Type** to 0 if the objective function f is **nonconvex** or if you have **no prior knowledge of the structure of the objective function** f .
 - Set **Type** to 1 if the objective function f is **convex**.
 - Set **Type** to 2 if the objective function f is **convex and piecewise linear**.
4. **options** is an *optional* input and is discussed in Section 7.

¹This is the link of a nonsmooth optimization lab.

5 Output of the BT-GS

The outputs of the **BT-GS** solver are stored in the **output** structure, which contains the following fields:

1. **output.f_end**: The lowest objective-function value encountered by the solver.
2. **output.x_end**: The computed approximation of a (local) minimizer or a stationary point.
3. **output.f_eval**: The total number of function evaluations.
4. **output.g_eval**: The total number of subgradient evaluations.
5. **output.cpu_time**: The elapsed CPU time.
6. **output.iters**: The total number of outer iterations.
7. **output.inner_iters**: The total number of inner iterations.
8. **output.f_rec**: The sequence of objective-function values generated during the optimization process.
9. **output.optimalty_certificate**: The smallest value of the stationarity measure observed during the optimization process.
10. **output.exit_flag**: The termination status of the solver, indicating the reason for termination as follows:
 - **exit_flag 1**: The solver terminated because the stationarity measure reached the prescribed tolerance.
 - **exit_flag 2**: The progress in the objective function and the iterates toward a stationary point became sufficiently small.
 - **exit_flag 3**: The objective-function value dropped below the value prescribed by the user.
 - **exit_flag 4**: The solver reached the maximum number of iterations.
 - **exit_flag 5**: The solver reached the maximum allowed run time.
 - **exit_flag 0**: The solver terminated due to a failure in the line search procedure.

6 Example 1

In this example, We intend to apply the BT-GS solver to the following nonconvex minimization problem:

$$\min f(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{x} \in \mathbb{R}^2,$$

where $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ is defined by

$$f(\mathbf{x}) = \max\{x_1^2 + (x_2 - 1)^2 + x_2 - 1, -x_1^2 - (x_2 - 1)^2 + x_2 + 1\}.$$

Since the nonsmooth region of f has a crescent-like shape, this function is referred to as the *Crescent* function. The global minimizer of this problem is $\mathbf{x}^* = [0, 0]$ with the optimal value $f(\mathbf{x}^*) = 0$.

First, we create a MATLAB function that, for a given $\mathbf{x} \in \mathbb{R}^2$, returns the function value $f(\mathbf{x}) \in \mathbb{R}$ and an arbitrary subgradient $\mathbf{g} \in \partial f(\mathbf{x})$. The function can be implemented as follows:

```

function [f, g]=Crescent(x)

f= max([x(1)^2+(x(2)-1)^2+(x(2)-1);-(x(1)^2)-(x(2)-1)^2+(x(2)+1)]);

if x(1)^2+(x(2)-1)^2+(x(2)-1)>=-(x(1)^2)-(x(2)-1)^2+(x(2)+1)

g=[2*x(1);0]+[0;2*x(2)-2]+[0;1];

else

g=[-2*x(1);0]+[0;-2*x(2)+2]+[0;1];

end

end

```

Notice: The first output of this function must be $f(\mathbf{x})$, while the second output must be an arbitrary subgradient $\mathbf{g} \in \partial f(\mathbf{x})$. The subgradient must be provided as a *column vector*.

Save the function in the same directory where the main **BT-GS** files are stored, ensuring that the file name matches the name of the function. Next, create a script file to call the **BT-GS** solver as follows:

```

clc

clear

x0=[3, 2];

Type=0;

output=BTGS(@Crescent,x0,Type);

```

Save this script file in the same directory as the main **BT-GS** files, using an arbitrary name, and then run the script to observe the results.

7 Options and Stopping Criteria

As mentioned above, **options** is an optional input to the **BT-GS** solver. It is a structure containing the following fields:

- **options.m:** This field controls the *number of sampled points* used to construct the initial polyhedral model of the objective function. It must be a *positive integer*. Its default value is defined as

$$\mathbf{options.m} = \begin{cases} 2n, & \text{if } n \leq 25, \\ n, & \text{if } 25 < n \leq 50, \\ \lfloor n/4 \rfloor, & \text{if } 50 < n \leq 100, \\ 10, & \text{if } 100 < n \leq 1000, \\ 5, & \text{if } 1000 < n \leq 2000, \\ 2, & \text{if } n > 2000, \end{cases}$$

where n is the dimension of the problem.

For challenging problems, a larger value of this field can be used, at the cost of increased storage requirements and CPU time.

Notice: If function and subgradient evaluations are computationally expensive, a smaller value of **options.m** can be used to reduce the computational cost.

- **options.max_iter:** This field specifies the maximum number of (outer) iterations allowed for the **BT-GS** solver. It must be a *positive integer*, and its default value is 5×10^4 .
- **options.max_time:** This field specifies the maximum allowed run time of the solver. It must be a *positive number*, and its default value is ∞ .
- **options.f_target:** The **BT-GS** terminates once the objective-function value becomes smaller than **options.f_target**. It can be any value in $[-\infty, \infty)$, and its default value is $-\infty$.

For academic comparisons, it is often desirable to terminate the solver once the relative error

$$\frac{|f(\mathbf{x}_k) - f^*|}{|f^*| + 1} \quad (2)$$

becomes smaller than a prescribed tolerance $\varepsilon > 0$, where f^* is the known optimal value. For this purpose, set

$$\mathbf{options.f_target} := f^* + \varepsilon(|f^*| + 1).$$

- **options.s_tol:** The **BT-GS** terminates when the current point satisfies the prescribed stationarity tolerance. It must be *nonnegative*, and its default value is 10^{-8} . A large value may cause the solver to terminate prematurely.
- **options.x_tol** and **options.f_tol:** The **BT-GS** terminates when the progress in *both* the iterates and the objective function becomes smaller than the corresponding tolerances. Both fields must be chosen from the interval $(0, 0.1)$, and their default value is 10^{-6} . Large values may cause the solver to terminate prematurely.
- **options.grad_eval:** The **BT-GS** supports three modes for supplying subgradients:

- **Mode 0:** The solver approximates subgradients from function values using a forward finite difference formula with a constant step size. To activate this mode set

$$\mathbf{options.grad_eval} = 0.$$

- **Mode 1:** The solver approximates subgradients from function values using a forward finite difference formula with a decaying step-size rule. To activate this mode set

$$\mathbf{options.grad_eval} = 1.$$

- **Mode 2:** The user supplies the subgradients directly, as demonstrated in Section 6. This is the *default mode*; therefore, no setting is required to use Mode 2. It is generally the most robust and fastest mode of the **BT-GS**, particularly for medium- and large-scale problems.

8 Example 2

In this example, we consider the following minimization problem:

$$\min_{\mathbf{x} \in \mathbb{R}^{50}} f(\mathbf{x}),$$

where $f : \mathbb{R}^{50} \rightarrow \mathbb{R}$ is given by

$$f(\mathbf{x}) = \max_{1 \leq i \leq 50} \{|x_i|\}. \quad (3)$$

This function, known as the *MAXL* function, has the global minimizer $\mathbf{x}^* = \mathbf{0}$ with $f(\mathbf{x}^*) = 0$. Moreover, f is convex and piecewise linear. Therefore, we set the input parameter **Type** to 2.

In this example, we employ the **BT-GS** using **Mode 0** for supplying subgradients. In this mode, the solver automatically approximates subgradients from function values using a forward finite-difference formula with a constant step size. Therefore, the user only needs to provide the function value. Accordingly, we create a MATLAB function that, for a given $\mathbf{x} \in \mathbb{R}^{50}$, returns only the function value $f(\mathbf{x}) \in \mathbb{R}$:

```
function f = MAXL(x)

f = max(abs(x));

end
```

Save this function in the same directory where the main **BT-GS** files are stored, ensuring that the file name matches the name of the function. We intend to terminate the solver once the relative error (2) becomes smaller than 10^{-5} . Since $f^* = 0$, we set

$$\text{options.f_target} = 10^{-5}.$$

The script for calling the **BT-GS** solver is therefore:

```
clc

clear

x0 = ones(50,1);

Type = 2;

options.grad_eval = 0;

options.f_target = 1e-5;

output = BTGS(@MAXL,x0,Type,options);
```

Save this script in the same directory as the main **BT-GS** files, using an arbitrary name, and then run the script to observe the results.

Notice: Since the user has supplied only f -values, **options.grad_eval=0** must be specified. If this setting is omitted, the solver uses the default Mode 2 and expects the user-supplied function to provide both f and g . In this case, the solver displays the following error message:

“The user-supplied function must provide both f and g since **options.grad_eval=2**.”

If the run is successful, the solver should terminate with **exit_flag=3**, indicating that the objective-function value has fallen below **options.f_target**. If the solver terminates with **exit_flag=2**, smaller values of **options.f_tol** and **options.x_tol** may be tried.

9 Smooth Minimization Problems

When the objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is smooth (i.e., continuously differentiable), the **BT-GS** can also be used effectively. In this case, $\partial f(\mathbf{x}) = \{\nabla f(\mathbf{x})\}$, for any $\mathbf{x} \in \mathbb{R}^n$. For *ill-conditioned* smooth optimization problems, the **BT-GS** can be employed using the same settings recommended for nonsmooth problems. For well-conditioned smooth problems, it is recommended to set

options.m=5 or **options.m=2.**

10 Troubleshooting

- **The algorithm terminates too early:** Try decreasing **options.x_tol**, **options.f_tol**, and **options.s_tol**. Also, check whether the initial point is already close to a stationary point.
- **The algorithm requires many iterations:** Try providing a better initial point.
- **Excessive run time:** If possible, use the **BT-GS** with the default Mode 2, which is generally the fastest mode (see Section 7). Make sure that the MATLAB function used to provide function and subgradient evaluations is efficiently implemented. The **BT-GS** may not be suitable for extremely large-scale problems, as it employs a relatively complex line-search procedure and solves a convex quadratic subproblem at each iteration.

11 Citation

If this software is used in academic research, publications, presentations, or other scholarly work, please cite the corresponding research article:

Morteza Maleknia, Majid Soleimani-damaneh, A bundle-trust method via gradient sampling technique for nonsmooth optimization using exact and inexact gradients, IMA Journal of Numerical Analysis, 2025;, draf087, <https://doi.org/10.1093/imanum/draf087>

12 Contact

BT-GS is a work of human effort. Like any human-developed software, it may contain bugs, imperfections, or other issues. Moreover, BT-GS has been developed without the use of artificial intelligence. We therefore greatly appreciate it if users report any bugs, unexpected behavior, or other issues they encounter. Such feedback helps us improve the reliability and performance of the software.

For questions, bug reports, or suggestions, please contact us at one of the following email addresses:

maleknia.morteza@gmail.com
m.maleknia@iut.ac.ir
m.soleimani.d@ut.ac.ir
soleimani_d@yahoo.com